FUNDACIÓN DE ESTUDIOS SUPERIORES FESC

TECHNICAL DOSSIER — SOFTWARE ARCHITECTURE AND DESIGN

Practical Guides No. 1 to 4 — GoApple Project

Project	GoApple — Web Commercial Management and Inventory System
Subject	Software Architecture and Design
Teacher	Ing. Robinson Damián Gómez Sánchez
Students	James Quintero Mosquera Giorgi Julián Ordóñez
Institution	Fundación de Estudios Superiores FESC
Campus	San José de Cúcuta, Colombia
Date	March 10, 2026

SOFTWARE ARCHITECTURE AND DESIGN
SAN JOSÉ DE CÚCUTA, COLOMBIA — 2026

GUÍA N°1 SOFTWARE ARCHITECTURE DEFINITION FOR GOAPPLE

Date	February 10, 2026
Authors	James Quintero Mosquera Giorgi Julián Ordóñez

Activity 1: Project Planning (Business Architecture)

This section defines the "what" and "why" of the GoApple system.

Project Title: *GoApple — Web Commercial Management and Inventory System*

Problem to solve: Currently, Apple product management (iPhone, iPad, MacBook, accessories) is done manually or in scattered spreadsheets. This causes:

- Loss of inventory control.
- Errors in sales recording.
- Lack of customer follow-up.
- Difficulty generating financial reports.
- Risk of data loss.

GoApple emerges as a centralized web solution that will allow managing products, customers, sales, and reports in an organized and secure manner.

Scope — 3 key processes:

1. Inventory Management

- Product registration (name, category, price, stock).
- Stock updates.
- Inflow and outflow control.
- Low stock alerts.

2. Sales Management

- Sales recording.
- Association of sold products.
- Automatic total calculation.
- Receipt generation.

3. Customer Management

- Customer registration.
- Purchase history.

- Contact information lookup.

Activity 2: Conceptual Architecture Modeling

The system structure is defined following the layered architecture pattern:

Presentation Layer (Frontend)

- Technologies: HTML5 (semantic structure), CSS3 (responsive design), JavaScript (form validation).
- Function: Interface used by Administrator and Seller roles.

Business Layer (Backend)

- Technology: PHP.
- Function: Contains the system logic (prevent sales without stock, calculate totals, validate customers, update inventory).

Data Layer (Persistence)

- Technology: MySQL.
- Main tables: products, customers, sales, sale_detail, users.

Physical directory structure:

```
/goapple
│
├── /assets   → Visual resources (Presentation)
│   ├── /css
│   └── /js
│
├── /views    → Views (Interface)
│   ├── login.php
│   ├── dashboard.php
│   ├── productos.php
│   ├── ventas.php
│   └── clientes.php
│
├── /classes  → Business Logic (PHP)
│   ├── Usuario.php
│   ├── Producto.php
│   ├── Cliente.php
│   ├── Venta.php
│   └── Detalle_Venta.php
│
├── /config   → Configuration and DB
│   └── database.php
│
├── /includes → Reusable components
│   ├── header.php
│   └── footer.php
│
└── index.php   → Entry point
```

Activity 3: Physical Architecture Definition

The environment where GoApple will run is documented:

Component	Proposed Specification
Web Server	Estándar con PHP/XAMPP
Database Engine	Relaciones complejas, integridad referencial
Scripting Language	Compatibilidad y seguridad reciente
Client Browser	Chrome / Edge
Operating System	Windows 10/11

Activity 4: Research — Design Patterns (Refactoring.Guru)

What is a design pattern? They are reusable and proven solutions to common problems in software development.

GoApple applies the MVC (Model–View–Controller) architectural pattern:

- Model → Classes Product, Customer, Sale.
- View → Files in /views.
- Controller → Logic connecting views with models.

Reuse: By separating inventory logic into Product.php, it could be reused in a mobile app, an API, or a physical POS system, without rewriting the logic.

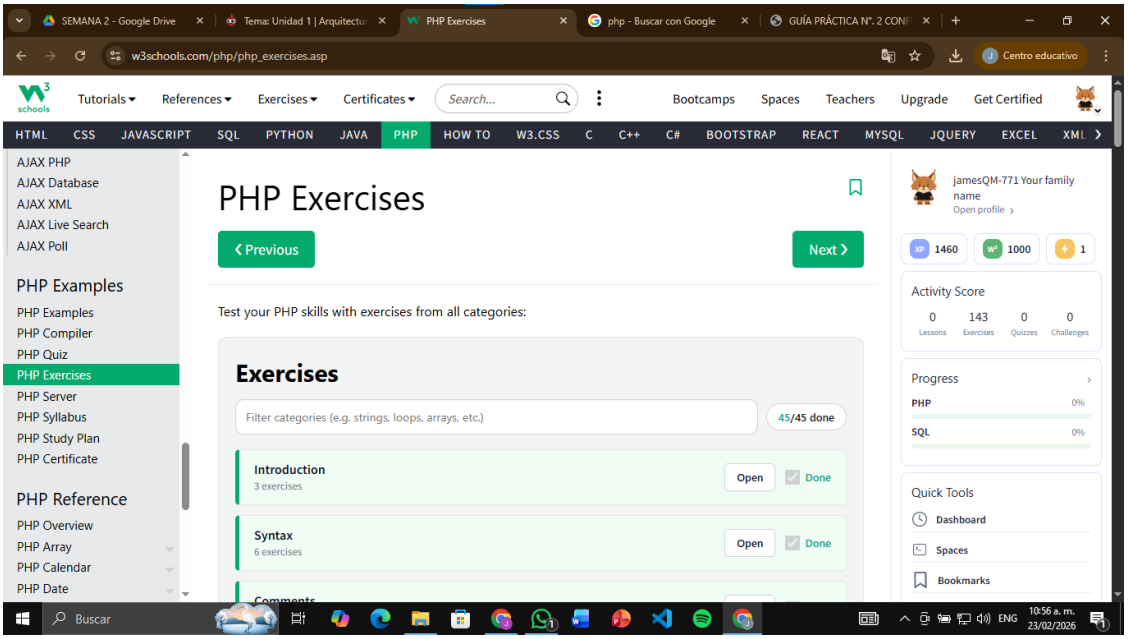
Evolution: If GoApple integrates a payment gateway, electronic invoicing, or advanced roles, only the business layer will need modification.

GUÍA N°2 PHP ENVIRONMENT AND LAYER CONFIGURATION FOR GOAPPLE

Date	February 17, 2026
Authors	James Quintero Mosquera Giorgi Julián Ordóñez

Activity 1: Intensive Training (W3Schools PHP Exercises)

W3Schools PHP Exercises were accessed and all practical exercises were completed: Syntax, Variables, Strings, Operators, If...Else, Switch, Loops, Functions, Arrays. The following shows evidence of exercises completed with "Correct" indicator:

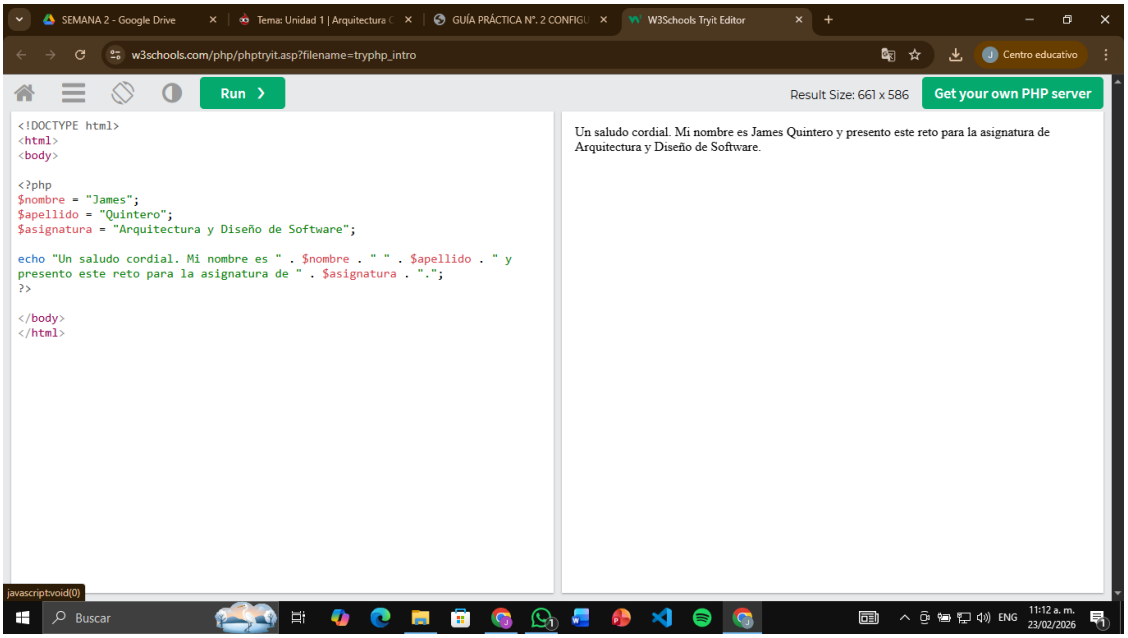


Activity 2: Code Writing Challenges (W3Schools PHP Tryit Editor)

Without any "autocomplete" aids, the following challenges were written from scratch in the Tryit Editor:

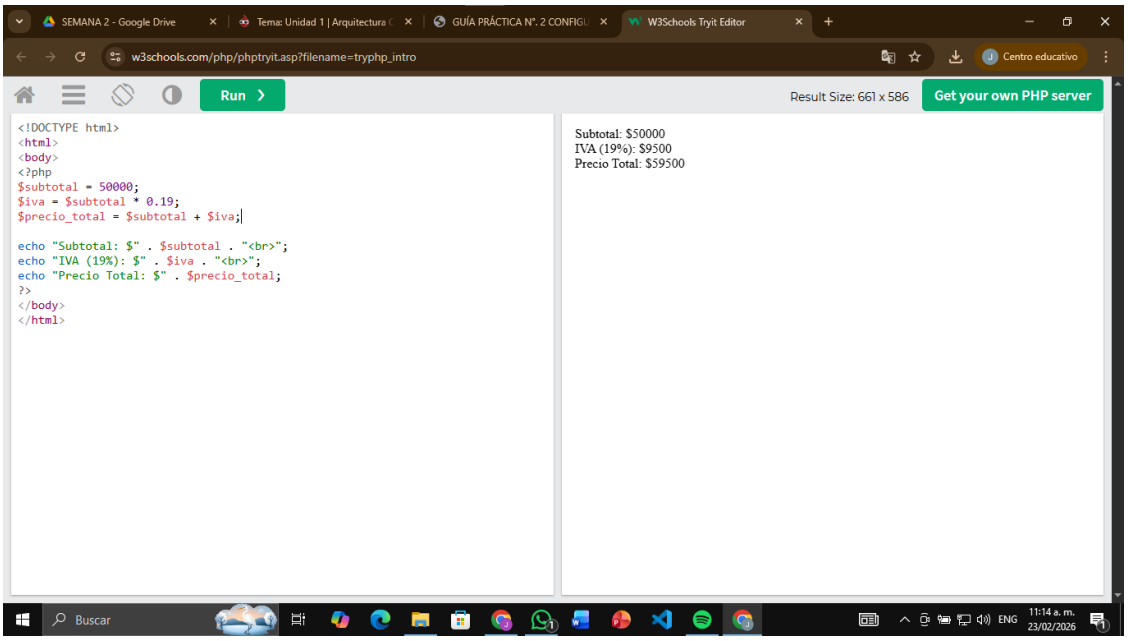
Challenge 1 — Variables: Formal greeting with concatenation

Variables for name, last name, and subject were declared and a formal greeting was displayed using string concatenation.



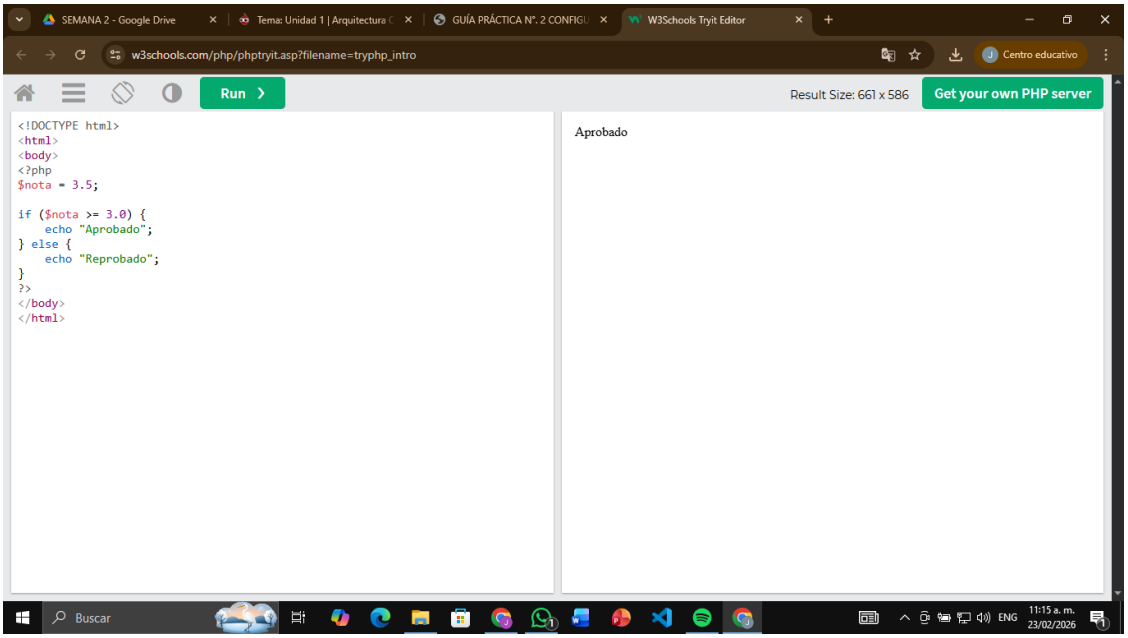
Challenge 2 — Operators: VAT Calculation (19%)

The variable $subtotal was defined, VAT at 19% and total price were calculated, displaying all three labeled values.



Challenge 3 — Conditional Logic: Pass / Fail

An if/else was used to evaluate the variable $nota. If ≥ 3.0 it shows "Passed", otherwise "Failed".



Activity 3: Technical Research — PHP Frameworks

What is a PHP Framework?

A PHP Framework is a development structure that provides an organized set of tools, libraries, and best practices to build web applications more quickly, securely, and in a structured way. Instead of starting from scratch, the developer uses a pre-built foundation that includes:

- Routing system (URL management).
- Database management.
- View templates.
- Session and authentication management.
- Protection against common vulnerabilities.
- Organization under the MVC pattern.

Laravel — Key Features

Feature	Description	GoApple Application
MVC Architecture	Clear separation of Model, View, and Controller.	Models: Products, Customers, Sales. Views: Admin panel. Controllers: Inventory logic.
Eloquent ORM	DB interaction using objects instead of raw SQL. Producto::all() instead of SELECT * FROM products.	Simplifies inventory management, reduces errors.
Built-in Authentication	Registration, login, roles, password recovery, protected routes.	Differentiates admin/seller, restricts critical modules.
Built-in Security	Protection against SQL Injection, CSRF, XSS. Password encryption.	Protects customer data, sales, and reports.

Feature	Description	GoApple Application
DB Migrations	Database version control.	Structural change control in team development.

Slim — PHP Micro-Framework

Slim is a lightweight and minimalist PHP micro-framework, designed to create APIs and small applications quickly and efficiently. Its features include: lightness (receive request → process → return response), simple routing, modularity, and high performance by consuming fewer resources.

GoApple Application: Slim would be useful if GoApple focused on exposing an API for a mobile app, or if the goal were to separate frontend and backend into a microservices model.

Reuse Criterion: How a Framework Helps

Criterion	Without Framework	With Framework
Authentication	Code from scratch (hours)	Built-in module (minutes)
Seguridad	Manual, inconsistent implementation	Automatic default protection
Code structure	Varies by developer	Standard convention (MVC)
Escalabilidad	Hard to add new modules	Independent reusable modules

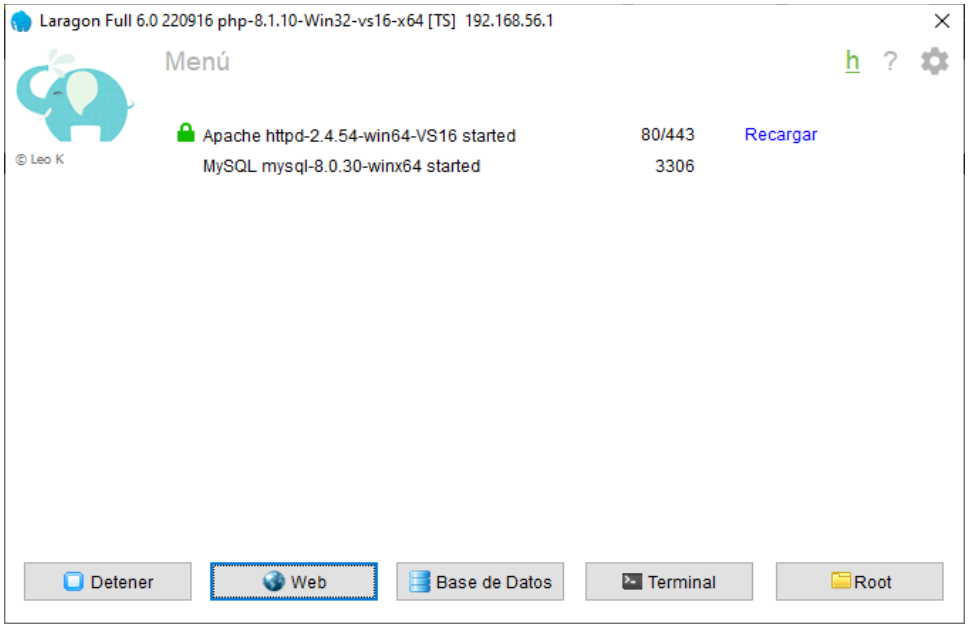
Decision: Native PHP for GoApple

For the GoApple project, in its initial phase, Native PHP was chosen for the following reasons:

- Deep architectural understanding: understanding layer interactions and manually implementing MVC strengthens technical learning.
- Full code control: design a custom architecture without external dependencies in the formative phase.
- Academic/formative project: forces manual implementation of validations, security, and structure, understanding what a framework automates.

Activity 4: Layer Architecture in VS Code / Laragon

Environment setup: Laragon was started (Apache/MySQL running) and the project folder was opened in VS Code at C:/laragon/www/.



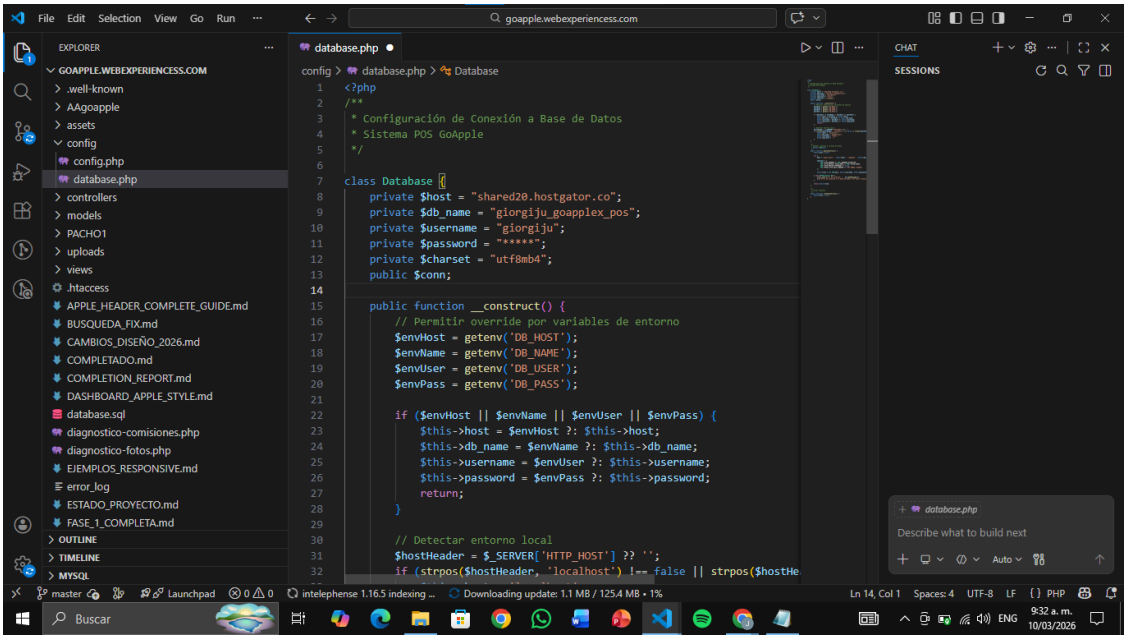
Physical structure manually created with folders representing the layered architecture: /app (Core and Data), /public (CSS, JS, images and index.php), /views (HTML interface files).

Activity 5: Integration Test — Layer Connection

To verify the folder structure works, the following file bridge was implemented:

- Step A: conexion.php was created in /app/Data/ with the database connection code.
- Step B: In index.php (root), include was used to call the previous file.
- Step C: The browser was accessed at localhost/project_name and both success messages were verified.

Evidence — Successful integration (VS Code + Browser):



GUÍA N°3 COMMUNICATION DIAGRAMMING AND EXTERNAL SYSTEMS FOR GOAPPLE

Date	February 24, 2026
Authors	James Quintero Mosquera Giorgi Julián Ordóñez

Activity 1: Interoperability Analysis and Data Standards

1. Technical definition of an API and its strategic role

An API (Application Programming Interface) is a set of rules and protocols that allows different software applications to communicate with each other. In the context of GoApple POS, the API acts as the mediating bridge: it allows the presentation layer (cashier's web browser) to consume business logic services (PHP server) without knowing its internal implementation, ensuring low coupling and facilitating horizontal scalability.

2. JSON standard analysis: advantages over XML

For communication between the GoApple POS web client and the server, HTTP with JSON (JavaScript Object Notation) will be used. JSON's competitive advantages over XML are:

Criterion	JSON	XML
Payload Size	Compact and lightweight	Verbose, larger size
Processing	Native in JavaScript (no libraries)	Requires external parser
Readability	High for both humans and machines	Complex with namespaces and attributes
HTTP Latency	Minimal (small payloads)	Higher (tag overhead)

3. Identified external services

Since GoApple POS handles sales registration and invoicing, two external agents are identified:

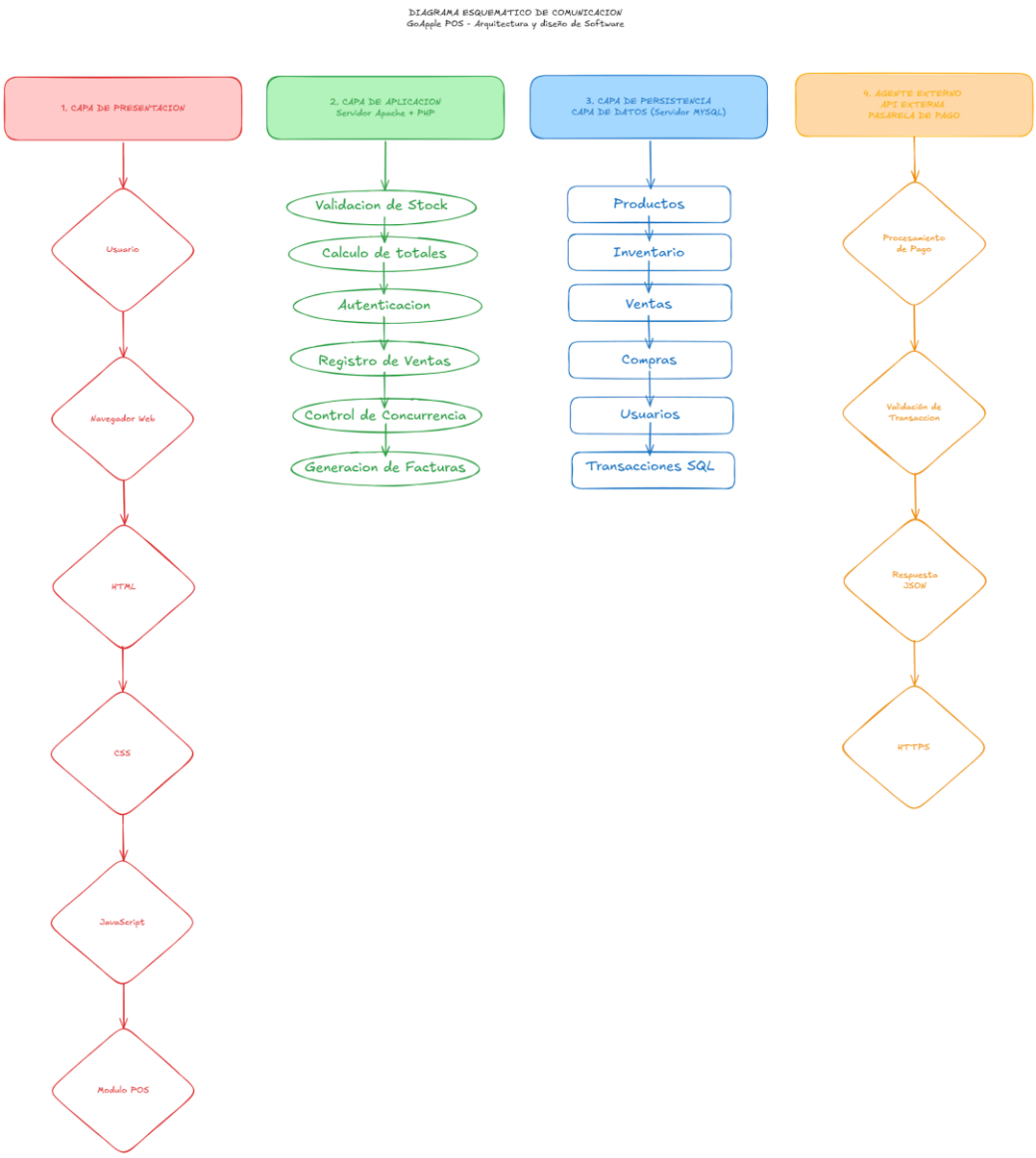
- Payment Gateway API (Stripe or Mercado Pago): to process electronic payments securely.
- MySQL engine as external persistence node: stores products, sales, purchases, and users with referential integrity.

Activity 2: Communication Schematic Diagram Modeling

Diagram created with a professional tool (Draw.io). It represents the technical interaction between the 4 system levels:

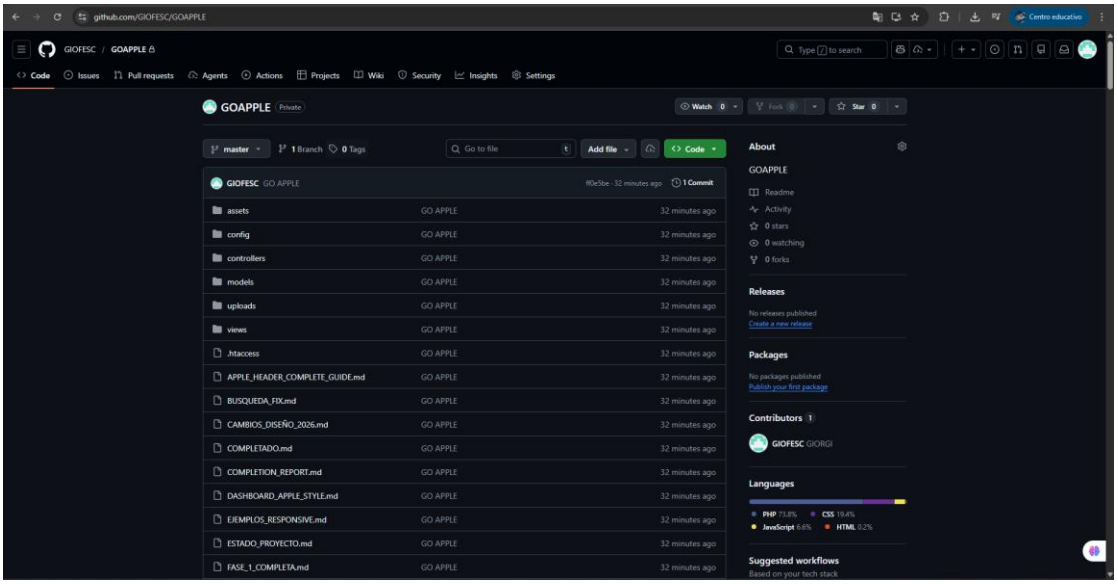
- Presentation Layer: User interface / Web browser.

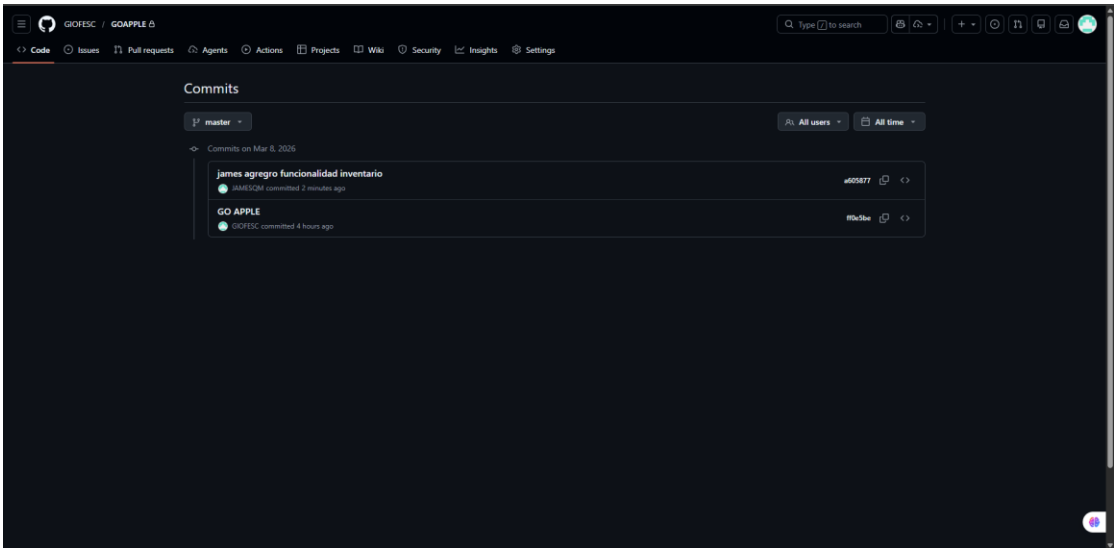
- Application Layer: Business logic on the PHP server.
- Persistence Layer: MySQL Database Server.
- External Agents: Payment Gateway API.



Activity 3: Configuration Management and Versioning (Git/GitHub)

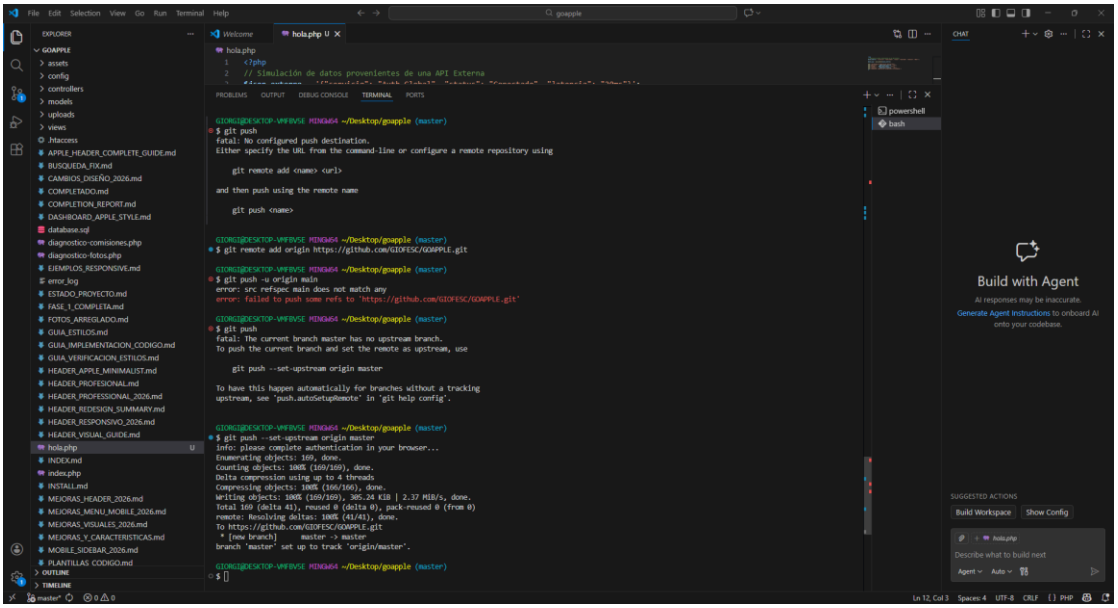
The Git version control system was implemented and the repository was linked to GitHub to manage the GoApple project source code collaboratively with change history.





Activity 4: Technical Implementation — Data Consumption Simulation

Data consumption simulation was implemented using fetch() and JSON, demonstrating communication between the presentation layer and the PHP server using the defined exchange standard.


File Edit Selection View Go Run Terminal Help
EXPLORER
GOAPPLE
assets
config
controllers
models
uploads
views
.htaccess
APPLE_HEADER_COMPLETE_GUIDE.md
BUSQUEDA_FIX.md
CAMBIOS_DISEÑO_2026.md
COMPLETADO.md
COMPLETION_REPORT.md
DASHBOARD_APPLE_STYLE.md
database.sql
diagnostico-comisiones.php
diagnostico-fotos.php
EJEMPLOS_RESPONSIVE.md
error_log
ESTADO_PROYECTO.md
FASE_1_COMPLETA.md
FOTOS_ARREGLADO.md
GUIA_ESTILOS.md
GUIA_IMPLEMENTACION_CODIGO.md
GUIA_VERIFICACION_ESTILOS.md
HEADER_APPLE_MINIMALIST.md
HEADER_PROFESIONAL.md
HEADER_PROFESSIONAL_2026.md
HEADER_REDESIGN_SUMMARY.md
HEADER_RESPONSIVO_2026.md
HEADER_VISUAL_GUIDE.md
hola.php
INDEX.md
index.php
INSTALL.md
MEJORAS_HEADER_2026.md
MEJORAS_MENU_MOBILE_2026.md
MEJORAS_VISUALES_2026.md
MEJORAS_Y_CARACTERISTICAS.md
MOBILE_SIDEBAR_2026.md
PLANTILLAS_CODIGO.md
OUTLINE
TIMELINE
Welcome
hola.php U
hola.php
1 <?php
2 // Simulación de datos provenientes de una API Externa
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git push
fatal: No configured push destination.
Either specify the URL from the command-line or configure a remote repository using
git remote add <name> <url>
and then push using the remote name
git push <name>
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git remote add origin https://github.com/GIOFESC/GOAPPLE.git
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/GIOFESC/GOAPPLE.git'
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git push
fatal: The current branch master has no upstream branch.
To push the current branch and set the remote as upstream, use
git push --set-upstream origin master
To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git push --set-upstream origin master
info: please complete authentication in your browser...
Enumerating objects: 169, done.
Counting objects: 100% (169/169), done.
Delta compression using up to 4 threads
Compressing objects: 100% (166/166), done.
Writing objects: 100% (169/169), 305.24 KiB | 2.37 MiB/s, done.
Total 169 (delta 41), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (41/41), done.
To https://github.com/GIOFESC/GOAPPLE.git
* [new branch] master -> master
branch 'master' set up to track 'origin/master'.
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$
powershell
bash
CHAT
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.
SUGGESTED ACTIONS
Build Workspace
Show Config
hola.php
Describe what to build next
Agent
Auto
Ln 12, Col 3
Spaces: 4
UTF-8
CRLF
PHP

GUÍA N°4 QUALITY ATTRIBUTES AND METRICS SPECIFICATION FOR GOAPPLE

Date	March 2, 2026
Authors	James Quintero Mosquera Giorgi Julián Ordóñez

Activity 1: Quality Attributes Matrix

Based on the quality models of Sommerville and Pressman, the following technical matrix is formalized with compliance thresholds for GoApple:

Attribute	Metric / Threshold	Implementation Strategy	Tool / Technique
Performance	Response time ≤ 1.5 s (full page on localhost)	FK column indexes. Optimized SQL queries with specific SELECT. PHP session cache.	índices MySQL
Security	0 XSS/SQL Injection vulnerabilities in critical forms	Validation with filter_var(). Prepared Statements with PDO. Sanitization with htmlspecialchars().	PHP PDO, htmlspecialchars(), OWASP Top 10
Reliability	Availability ≥ 99% Mean time between failures > 30 days	try/catch exception handling. Error logs in /logs/. MySQL transactions (BEGIN/COMMIT).	error_log(), PDO::beginTransaction(), XAMPP
Usability	≤ 3 clicks to complete a sale or registration	Responsive interface with CSS Grid/Flexbox. Clear error messages. Real-time validation with JS.	CSS nativo, JavaScript validación
Maintainability	Comment coverage ≥ 70% in model classes	Strict MVC separation. PSR-12 naming. PHPDoc documentation in models.	PSR-12, PHPDoc, /models /views /controllers
Scalability	Support ≥ 10 concurrent users without degradation	Stateless PHP architecture. DB / logic separation. Possibility to migrate to external VPS.	Apache mod_php, MySQL connection pooling

1.1 Performance

GoApple operates in a PHP/XAMPP environment where response time is critical for the cashier experience. The following metrics are established:

- Initial dashboard load: ≤ 1.5 seconds on localhost.
- Complete sale registration: ≤ 2 seconds.

- Inventory query (100–500 products): ≤ 0.8 seconds.

Strategies: PHP opcache for bytecode caching, indexes on frequently searched columns, SELECT queries with specific fields instead of SELECT *.

1.2 Security

- Input validation with filter_var() and filter_input() on all server routes.
- SQL Injection prevention using Prepared Statements with PDO.
- Sanitization with htmlspecialchars() to prevent XSS in views.
- Role-based access control: administrator and seller with PHP sessions verified on each request.

1.3 Reliability and Usability

Reliability — Fault tolerance mechanisms:

- Exception handling with try/catch blocks in all database operations.
- MySQL transactions (BEGIN / COMMIT / ROLLBACK) in multi-table operations.
- Log system in /logs/error.log with critical error recording via error_log().

Usability — User experience criteria:

- Navigation in maximum 3 clicks from dashboard to completing any operation.
- Clear feedback messages for each action (success, error, warning).
- Responsive interface on tablets and desktop PCs.
- Real-time validation with JavaScript before server submission.

Activity 2: Prospective Analysis — The "Cost of Change"

GoApple's MVC architecture significantly reduces the technical and economic impact of major changes. The most likely modification scenarios are evaluated:

Change Scenario	Affected Layer	Impact without MVC	Impact with MVC	Estimated Hours
MySQL → PostgreSQL migration	Data Layer	High: rewrite all query logic	Low: only modify database.php and models	4–8 h
Apache → Nginx change	Infrastructure	Medium: review .htaccess routes	Minimal: adjust server config	1–2 h
Electronic invoicing module	Business Layer	High: couple with existing code	Low: new controller + model	6–12 h
Complete frontend redesign	Presentation Layer	High: changes in every PHP script	Minimal: only /views/ files	3–6 h

Change Scenario	Affected Layer	Impact without MVC	Impact with MVC	Estimated Hours
OAuth 2.0 authentication	Business + Config	High: cross-cutting changes	Low: middleware in /Core/	4–8 h

2.1 Main Scenario: MySQL to PostgreSQL Migration

With GoApple's MVC architecture, the change is limited to:

- Modify database.php in /config/ to change the DSN from mysql: to pgsql:.
- Adapt models in /models/ that use MySQL-specific syntax (e.g.: AUTO_INCREMENT → SERIAL).
- The presentation layer (/views/) and controllers (/controllers/) remain unmodified.

Economic impact estimate: monolithic system ~40–80 hours; GoApple MVC architecture ~4–8 hours. Change cost reduction: 80–90%.

Activity 3: Validation Layer Implementation (Validator.php)

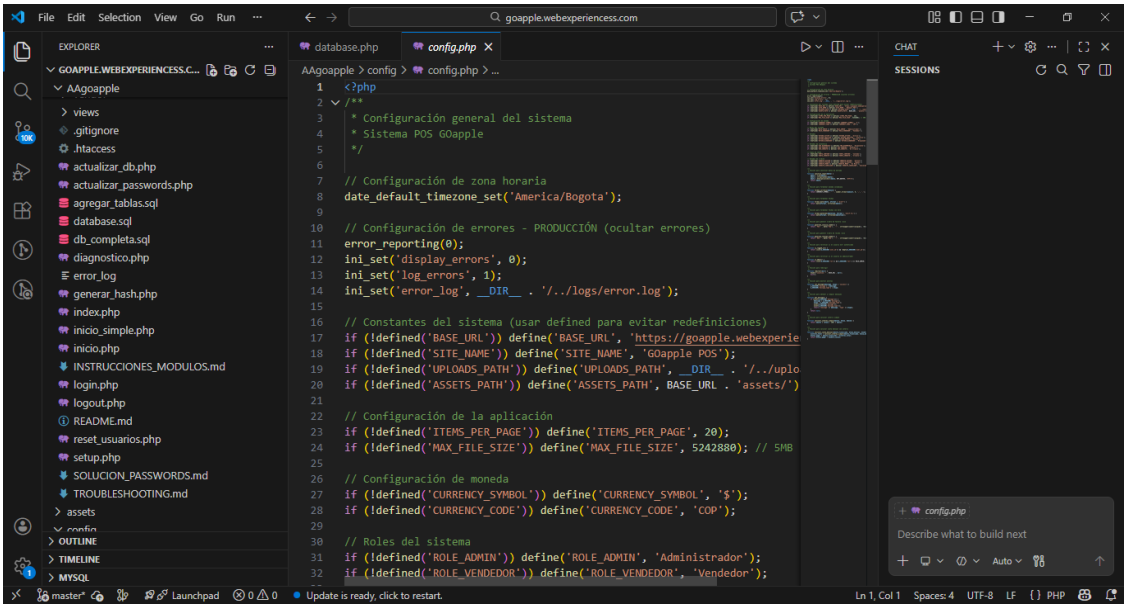
The Validator.php component is developed inside the /app/Core/ directory of the GoApple project. This component intercepts input data before processing in the business layer, applying native PHP filters.

3.1 Location in the Architecture

Path: /goapple/app/Core/Validator.php

- Acts as middleware between the HTTP request and the business layer.
- Invoked from controllers before passing data to models.
- Centralizes all validation logic, avoiding code duplication.

3.2 Source Code — VS Code Evidence



3.3 Component Methods

Method	PHP Filter	Description
texto()	FILTER_SANITIZE_SPECIAL_CHARS	Validates names, descriptions, and free text fields.
entero()	FILTER_VALIDATE_INT	Validates quantities, stock, numeric IDs.
decimal()	FILTER_VALIDATE_FLOAT	Validates prices, sales amounts, and credits.
email()	FILTER_VALIDATE_EMAIL	Validates customer and system user emails.
telefono()	preg_match — regex	Validates Colombian cell phones (10 digits, starting with 3).
tipoVenta()	in_array() estricto	Restricts sale type to allowed values.
sinInyeccion()	stripos() — lista negra	Detects dangerous SQL Injection patterns.

Activity 4: Architectural Dossier Consolidation

All technical documentation generated during the semester is systematically organized. This dossier constitutes the complete technical portfolio of the GoApple project and will be the main input for the in-person defense.

4.1 Technical Portfolio Index

N°	Section	Description	File
1	Practical Guide No.1	Project planning, conceptual and physical architecture, MVC patterns	Guia_N1_GoApple.docx

N°	Section	Description	File
2	Practical Guide No.2	PHP training, code challenges, frameworks, VS Code architecture	Guia_N2_GoApple.docx
3	Practical Guide No.3	APIs, JSON vs XML, communication diagram, Git/GitHub, data simulation	Guia_N3_GoApple.docx
4	Practical Guide No.4	Quality matrix, change cost, Validator.php, technical dossier	Guia_N4_GoApple.docx
5	Communication Diagram	Visual representation 4 layers: Presentation → Business → Data → APIs	Diagrama_Comunicacion.png
6	Directory Structure	Physical GoApple project tree with layer responsibilities	Estructura_Fisica.png
7	Code — Validator.php	Validation component in /app/Core/. VS Code screenshots	Validator_VSCode.png
8	Database	SQL script: products, customers, sales, sale_detail, users, credits	database.sql

Final Note — Preparation for Defense

This unified document completes the architectural dossier of the GoApple project for the Software Architecture and Design course (FESC). For the in-person defense you should have: (1) the functional project in XAMPP/Laragon with the MVC structure implemented, (2) this portfolio in digital or printed format, (3) the communication diagram from Guide No.3, and (4) a demonstration of the Validator.php component running.